

Classic Burger

Jujur saya bingung nama tim nya apa
Cryptography

Write-up Penyelesaian

1. Jadi di challenge ini kita diberikan 2 file yaitu file yaitu [enkripsi.py](#) dan key.txt, maka disini saya menganalisis isi kedua file tersebut di kode [enkripsi.py](#) terdapat sebuah kode berikut.

```
C:\> Users > ariq > Downloads > PY enkripsi.py > ...
1  from Crypto.Util.number import getPrime, bytes_to_long
2
3
4  with open('flag.txt', 'rb') as f:
5      flag = f.read()
6
7  p = getPrime(1024)
8  q = getPrime(1024)
9  r = getPrime(1024)
10
11  n1 = p * q
12  n2 = p * r
13  n3 = q * r
14
15  moduli = [n1, n2, n3]
16
17  e = 65537
18  c = bytes_to_long(flag)
19
20  for n in moduli:
21      c = pow(c, e, n)
22
23  with open('Key.txt', 'w') as f:
24      f.write(f'n1: {n1}\n')
25      f.write(f'n2: {n2}\n')
26      f.write(f'n3: {n3}\n')
27      f.write(f'e: {e}\n')
28      f.write(f'c: {c}\n')
```

Di kode tersebut “RSA dengan tiga lapisan” dan menyimpan ke Key.txt. Pertama, ia membaca isi flag.txt sebagai bytes lalu mengubahnya menjadi bilangan bulat M (dengan `bytes_to_long`). Setelah itu program membangkitkan tiga bilangan prima 1024-bit: p, q, r . Dari tiga prima ini disusun tiga modul RSA yang saling beririsan: $n1 = pq, n2 = pr$, dan $n3 = qr$. Eksponen publiknya dipakai nilai standar $e = 65537$. Proses enkripsinya berlapis: mulai dari $c \leftarrow m^e \bmod n_1$, hasilnya dienkripsi lagi $c \leftarrow c^e \bmod n_2$, lalu lagi sekali $c \leftarrow c^e \bmod n_3$. Jadi ciphertext akhir adalah hasil pemangkatan berulang dengan modulo yang berbeda secara beruntun (tiga “lapis”). Terakhir, program menulis semua parameter- n_1, n_2, n_3, e, c - ke Key.txt. Ciri penting susunan n_1, n_2, n_3 seperti ini adalah pasangan modul berbagi faktor prima, sehingga $\gcd(n_1, n_2) = p$, $\gcd(n_1, n_3) = q$, $\gcd(n_2, n_3) = r$.

Lalu ketika saya buka isi file key.txt isi nya ada nilai variable dari rsa.

```
n1=
1269489119287636006442497089981944590461187272925482598638845608103849360295317799596302663822868762434409149445245922496324443867182878481403965827849511780286318219716231439931050422666406736295339176085188555
199580433682065311981283779984604108863727736284235583212686940632715875948862437475947818633022809799927973239856442762072647616230392864170072861803201019612168065047571280441794074858608568101122861721
778428984043698257558397515644595369584614252199784980294373184715787241061924452319722306137848898578800957675114029358173852511797075938563773772726289788978316904438921224515181170913990697
n2=
12455966510875493651140836764560558263736945066314511071372305222622952955669892538285142466241324749324837507899034971489867300764093837131973329829590349766547573457010891352776032594565579624052688010847453
607104694248916809830211081071224229828345889985804787062627183340233615849458701577880729723999491238331736957212387869230767634465394390686668267282077293879381088223897864141095958173446533544034110821921
79319645071043695119944676383659329367874400878899935806442183262329756258551768273785996070926861735656834909898338291431726646628048911422598105177727898402618582893874649356549051846348412823
n3=
13036928391328671973348588613587368376477190943705248497199171984764231739874636274608897976328773642098617057833383211348453711731240451016340648154375270687275362496603878438582549302559987150403899496235917
419721458192197759226245074921882910154542344426977784106869216436394061831891523276764029990227558283297408301818669902523708005176973003098019894744634857272154408301116255822408092324290141697819928528060
360784550490184194796957815265837868835387391170937468750201358483610951133154598512422218315496881099192137459740357649354924842940582828321437428427106258226030497510715596110270987615322169407
e= 65537
c=
1065680742159365639460204077287813478846006841205437027036814581637604492097913804443906976621550730054338847300928479930236679812578917440061738843400533882863939036402192545553845409827286167972889677562910598
8723401728531216198250675645819556446523338322326777153356807218554912646547859416396915828092171662309310066182081108345935283513382910937151086696414382841606954848581105302685802412313327793433060051792478603
68002967401088425376322422159479798966723897955397400726052822560080723555592613782876651579558327946816279979589951567394611610484348773754605330582791115212743353492466231559059168974433603090
```

2. Langkah selanjutnya saya disini Baca parameter & sanity check

Saya parse n_1, n_2, n_3, e, c dari key.txt.

Cek cepat:

- bitlength ketiga $n_i \sim$ ini ~ 2047 bit $\rightarrow$ ukuran RSA 2048-an, masuk akal.
- Coba cari pairwise gcd:
 - $gcd(n_1, n_2), gcd(n_1, n_3), gcd(n_2, n_3)$.
 - Hasil awal = 1 semua $\rightarrow$ tidak mungkin jika benar-benar pq, pr, qr .
 $\Rightarrow$ Ada pra-proses (sesuai hint).

3. Lalu disini saya memanfaatkan Hint yaitu “carriage number 4”

“Carriage 4” sangat kuat mengarah ke Rail-Fence Cipher dengan 4 rails. “Proses lain sebelum proses sesungguhnya” = decrypt rail-fence dulu sebelum proses RSA.

Observasi di konten key.txt:

Label $n1=...$, $n2=...$, $n3=...$ tampak normal, tapi angka di dalamnya bisa saja disusun ulang.

Uji beberapa transformasi ringan atas string angka:

- reverse, interleave/deinterleave mod-4, rail-fence decrypt (4 rails)

4. Maka disini saya memiliki ide untuk Uji transformasi & temukan kombinasi yang “mengembalikan” struktur RSA overlap.

Saya lakukan brute ringan pada tiap nin_in: terapkan beberapa transformasi, lalu tes GCD antar kandidat.

- Kriteria sukses: semua GCD pasangan $\geq \sim 512$ bit (menandakan “prime besar” yang sama).
- Hasil terbaik:

- $n_1n_1n_1 \rightarrow$ rail-fence decrypt (4 rails)
- $n_2n_2n_2 \rightarrow$ apa adanya (id)
- $n_3n_3n_3 \rightarrow$ rail-fence decrypt (4 rails)
- Dengan transformasi di atas:

$p = \gcd(n_1, n_2), q = \gcd(n_1, n_3), r = \gcd(n_2, n_3)$. semuanya **1024 bit** $\rightarrow$ pas!

5. Lalu selanjutnya saya disini rekonstruksi kunci privat per lapisan.

Setelah dapat p, q, r :

- $n_1 = pq, \phi_1 = (p - 1)(q - 1) \rightarrow d_1 \equiv e^{-1} \pmod{\phi_1}$
- $n_2 = pr, \phi_2 = (p - 1)(r - 1) \rightarrow d_2 \equiv e^{-1} \pmod{\phi_2}$
- $n_3 = qr, \phi_3 = (q - 1)(r - 1) \rightarrow d_3 \equiv e^{-1} \pmod{\phi_3}$

6. Lalu disini saya mengurutkan dekripsi

Dari kode “RSA 3 lapis” $\rightarrow$ plaintext m di enkripsi secara beruntun (misal n_1 lalu n_2 lalu n_3).

Maka dekripsi harus **dibalik**:

- $m_3 = c^{d_3} \pmod{n_3}$
- $m_2 = m_3^{d_2} \pmod{n_2}$
- $m_1 = m_2^{d_1} \pmod{n_1} \rightarrow$ plaintext

7. Maka langkah terakhir disini saya membuat sebuah script dari hasil analisis diatas agar mendapatkan flag nya dengan bentuk kode berikut.

```
C:\Users\ariq> Downloads > PY try_pw.py > ...
1 import re
2 from math import gcd
3 from Crypto.Util.number import inverse, long_to_bytes
4
5
6 def railfence_decrypt_digits(num_str: str, rails: int = 4) -> str:
7     s = ''.join(ch for ch in num_str if ch.isdigit())
8     if rails <= 1 or len(s) <= rails:
9         return s
10
11     pattern = list(range(rails)) + list(range(rails - 2, 0, -1))
12     L = len(s)
13
14     counts = [0] * rails
15     for i in range(L):
16         counts[pattern[i % len(pattern)]] += 1
17
18     rails_buf = []
19     idx = 0
20     for c in counts:
21         rails_buf.append(list(s[idx:idx + c]))
22         idx += c
23
24     pos = [0] * rails
25     out = []
26     for i in range(L):
27         r = pattern[i % len(pattern)]
28         out.append(rails_buf[r][pos[r]])
29         pos[r] += 1
30     return ''.join(out)
31
32
33 def read_key_text():
34     paths = ["Key.txt", "key.txt", "./Key.txt", "./key.txt"]
35     for p in paths:
36         try:
37             return open(p, "r", encoding="utf-8").read()
38         except OSError:
39             continue
40     raise FileNotFoundError("Tidak menemukan Key.txt/key.txt di folder skrip.")
41
42 def grab_int(label: str, text: str) -> str:
43
44     m = re.search(rf"{label}\s*[:=]\s*([0-9]+)", text)
45     if not m:
46         raise ValueError(f"Gagal menemukan {label} di Key.txt")
47     return m.group(1)
```

8. Dan ketika dijalankan mengeluarkan output berikut.

```
30     return ''.join(out)
31
32
33     def read_key_text():
34         paths = ["Key.txt", "key.txt", "./Ke
35         for p in paths:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORT

```
PS C:\Users\ariq\Documents\srifotonfinals> & C:/
SRIFOTON{R0t1_D4g1nG_R0t1_ar3rs91d}
PS C:\Users\ariq\Documents\srifotonfinals>
```

9. Maka Flag dari chall tersebut adalah : SRIFOTON{R0t1_D4g1nG_R0t1_ar3rs91d}